

Amortized Analysis

May 15, 2026

Problem 1

We maintain a set S of real valued items that supports the following two operations:

- $\text{Insert}(S, x)$: insert item x into set S
- $\text{DeleteLargerHalf}(S)$: delete the $\lceil |S|/2 \rceil$ largest valued items from S .

The Insert operation allows duplicate values to be inserted into S . So, item values in S are not necessarily distinct. Briefly explain how to implement these two operations to run in $O(1)$ amortized time.

Solution

We use linked-list to sort S , and insert new element at the head of the linked list.

Amortized equation:

$$\text{amortized cost} = \text{actual cost} + \text{accounting cost}$$

Actual cost:

- $\text{Insert}(S, x)$: 1
- $\text{DeleteLargerHalf}(S)$: tn . find the $n+1-\lceil |S|/2 \rceil$ smallest elements x in S , and delete any element larger than or equal to x .

Accounting scheme:

- $\text{Insert}(S, x)$: $2t$
- $\text{DeleteLargerHalf}(S)$: $-tn$

Therefore the amortized costs of two operations are given as follows:

- amortized cost of $\text{Insert}(S, x) = 1 + 2t = O(1)$
- amortized cost of $\text{DeleteLargerHalf}(S) = 0$

We have to prove that the sum of the accounting cost is not less than 0 (i.e., positive). For any $\text{DeleteLargerHalf}()$, assume S has n elements at this time and the operation has $-tn$ accounting cost. For the deleted elements x by this operation, the number of them is larger than $n/2$, and each of them has a $2t$ accounting cost when it is inserted. (Note that for any inserted element x , it has at most one deletion in $\text{DeleteLargerHalf}()$.) Therefore the total accounting cost is not less than 0.

Problem 2

Describe how to implement a queue using two stacks and $O(1)$ additional memory, so that the amortized time for any **enqueue** or **dequeue** operation is $O(1)$. The only access you have to the stacks is through the standard subroutines **Push** and **Pop**.

Solution

We set the **enqueue/dequeue** points of the queue as the start/end of the queue. Briefly, we employ two stacks (**Stack_s** and **Stack_e**) to sort the whole elements into two parts, i.e., one part from the start point and the other part from the end point.

```

function enqueue( $x$ )
    Stacke.Push( $x$ )
end function
function dequeue
    if Stacks.IsEmpty() = True then
        while Stacke.IsEmpty()  $\neq$  True do
             $k$  = Stacke.Pop()
            Stacks.Push( $k$ )
    
```

```
        end while
    end if
    return Stacks.Pop()
end function
```

We analyze the following implementation, for any elements x in the queue,

- first, x is pushed in **Stack_e** (i.e., **enqueue**(x));
- second, x is popped from **Stack_e** and pushed in **Stack_s**;
- finally, x is popped from **Stack_s**.

On the whole, for x , we implement two **Pushes** and two **Pops**. We move all these four operators into the **enqueue** operation of x .